📦 Component Library v1 — Karzalay

The primary design system elements and reusable building blocks for the **Karzalay** product. Strictly engineered using design tokens with **zero hardcoded values** and streamlined to exclusively support the selected **Academic Amethyst** global theme.

🏗️ Global Design Tokens Setup

- **Design System Profile:** Academic Amethyst (Royal Amethyst Purple theme with Lora serif headings, Inter body text, and a Lavender Alabaster background)
- **Integration Mechanics:** Uses CSS custom variables mapped inside `:root`. All elements across all routes naturally inherit these styling tokens.

🎨 Theme Token Variables

```
:root {
  /* Spacing Scale (Base unit 8px) */
  --spacing-4: 4px;
  --spacing-8: 8px;
  --spacing-12: 12px;
  --spacing-16: 16px;
  --spacing-24: 24px;
  --spacing-32: 32px;
  --spacing-48: 48px;
  --spacing-64: 64px;

  /* Universal Semantics */
  --color-error: #EF4444;
  --color-success: #10B981;
  --color-warning: #F59E0B;

  /* Global Theme Profile: Academic Amethyst */
  --theme-name: "Academic Amethyst";
  --color-primary: #7C3AED;       /* Royal Amethyst Purple */
  --color-secondary: #D946EF;     /* Vibrant Plum */
  --color-background: #FAF8FC;    /* Lavender Alabaster */
  --color-surface: #FFFFFF;
  --color-border: #EAE5F3;
  --color-border-hover: #7C3AED;
  --color-text-primary: #1E1B29;
  --color-text-secondary: #5C5870;
  --color-accent-bg: #F3E8FF;

  /* Typography */
  --font-family-heading: var(--font-lora), Georgia, serif;
  --font-family-body: var(--font-inter), sans-serif;

  --font-size-h1: clamp(2.5rem, 5vw, 4rem);
  --font-size-h2: clamp(1.8rem, 3.5vw, 2.75rem);
  --font-size-h3: clamp(1.4rem, 2.5vw, 1.75rem);
  --font-size-body-large: 1.125rem;
  --font-size-body: 1rem;
  --font-size-caption: 0.875rem;
  --font-size-label: 0.75rem;

  /* Layout & Animations */
  --radius-card: 8px;
  --radius-btn: 6px;
```

```
  --border-weight: 1px;
  --border-style: solid;
  --shadow-card: 0 2px 8px rgba(124, 58, 237, 0.02);
  --shadow-hover: 0 10px 24px rgba(124, 58, 237, 0.05);
  --transition-speed: 0.35s;
  --transition-curve: cubic-bezier(0.4, 0, 0.2, 1);
}
```

🔘 1. Button Component

A premium trigger element supporting variants, sizing options, and responsive interactions. No hardcoded sizes or colors.

Props Accepted

Prop Name	Type	Default	Description
`variant`	`'primary' | 'secondary' | 'danger' | 'ghost'`	`'primary'`	Visual style preset to apply.
`size`	`'sm' | 'md' | 'lg'`	`'md'`	Padding and font size configuration scale.
`disabled`	`boolean`	`false`	Standard HTML disabled state. Disables hover animations and lowers opacity.
`children`	`React.ReactNode`	—	Text or elements inside the button.
`className`	`string`	`""`	Extra CSS classes to append.

Visual Variations & States

1. **Primary (Filled):** Filled brand color background (`var(--color-primary)`). On hover, scales up smoothly and applies high-elevation shadow.
2. **Secondary (Outlined):** Subtle bordered outline. Adapts dynamically to card borders.
3. **Danger (Red):** Action indicator for critical tasks. Adapts dynamic errors.
4. **Ghost (Text Only):** Renders without borders or fills, blending cleanly into parent blocks. Soft accent fill on hover.

React Implementation

```
import React from "react";
import styles from "./Button.module.css";

export interface ButtonProps extends React.ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: "primary" | "secondary" | "danger" | "ghost";
  size?: "sm" | "md" | "lg";
  children: React.ReactNode;
}

export const Button = React.forwardRef<HTMLButtonElement, ButtonProps>(
  ({ variant = "primary", size = "md", children, className = "", ...props }, ref) => {
    const classNames = [
      styles.button,
      styles[variant],
      styles[size],
      className
    ].filter(Boolean).join(" ");

    return (
      <button ref={ref} {...props} disabled={loading || props.disabled} aria-busy={loading}>
        {children}
      </button>
    );
  }
);

Button.displayName = "Button";
```

v1.0.0 Stable Academic Amethyst

Karzalay Core Components

The primary building blocks of the Karzalay ecosystem. Engineered strictly with the Academic Amethyst design system tokens, maintaining absolute visual elegance.

Button Component

A premium trigger element supporting variants, sizing options, and responsive interactions. No hardcoded sizes or colors.

Variants

Primary Filled

Primary Variant

Secondary Outline

Secondary Variant

Danger Action

Danger Variant

Ghost Trigger

Ghost Variant

Figure 1.1: Button Variants (Primary, Secondary, Danger, Ghost) under hover states

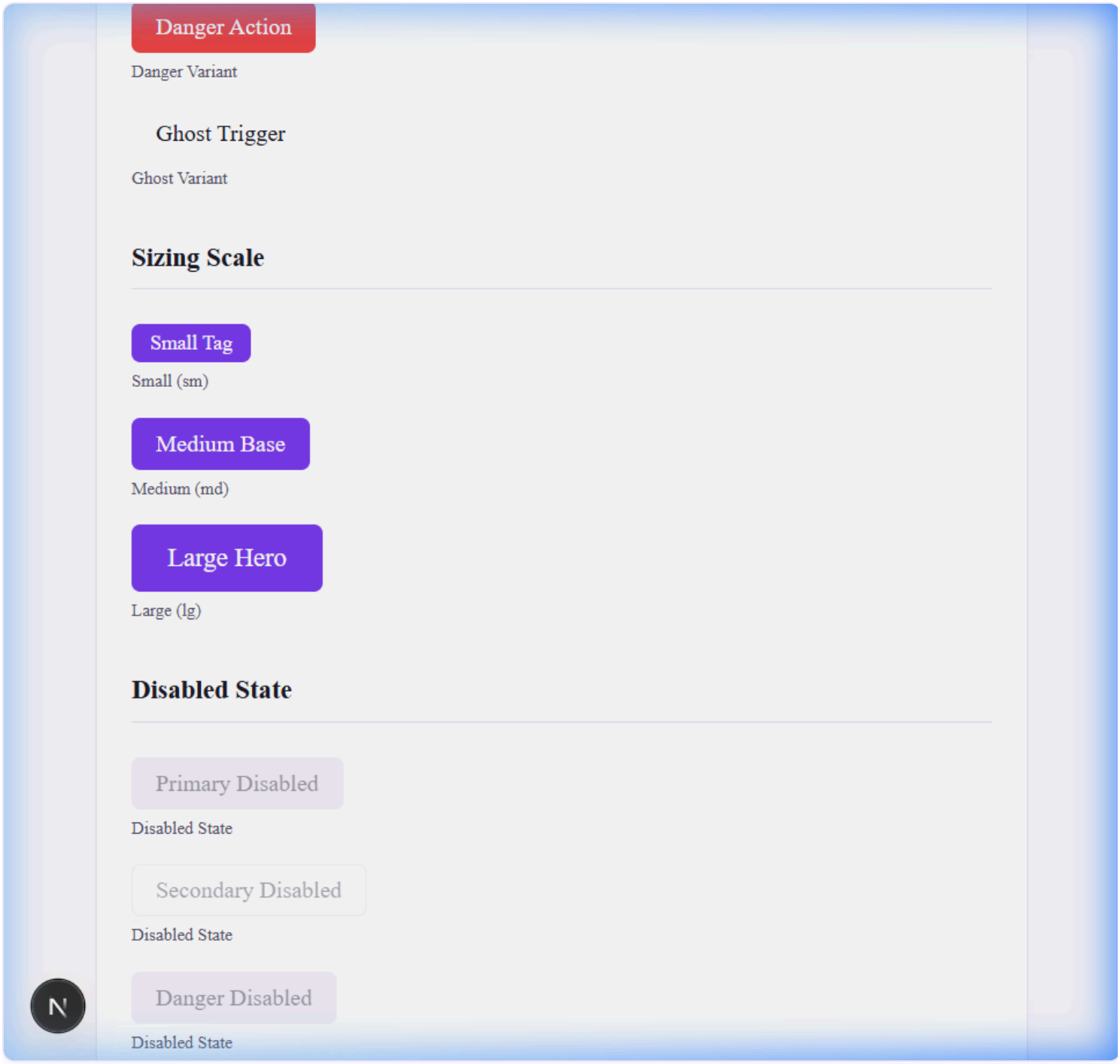

Figure 1.2: Button Sizing Scales (Small, Medium, Large) and Disabled state treatments

📥 2. Input Component

Standard text, email, and password fields supporting labels, hover highlights, error alerts, and disabled states.

Props Accepted

Prop Name	Type	Default	Description
`label`	`string`	—	Field title label displayed above input box. Uses typography tokens.
`error`	`string`	—	Validation error message displayed below. Triggers error red borders.
`type`	`'text' | 'email' | 'password'`	`'text'`	Standard HTML input field type.
`disabled`	`boolean`	`false`	Disables input interactions, overlays background, and dims text.
`placeholder`	`string`	—	Default text visible inside input when empty.
`className`	`string`	`""`	Extra CSS classes to append.

Visual Variations & States

1. **Default State:** Sleek background matching token boundaries with light grey borders.
2. **Focused State:** Focus borders trigger an outline transition to the primary brand amethyst color with an active glow shadow.
3. **Error State:** Outline colors switch to the semantic warning/error red (`var(--color-error)`), and error details are displayed below the input.
4. **Disabled State:** Opacity is reduced to `0.5`, background inputs block interaction, and cursor changes to `not-allowed`.

React Implementation

```
import React, { useId } from "react";
import styles from "./Input.module.css";

export interface InputProps extends React.InputHTMLAttributes<HTMLInputElement> {
  label?: string;
  error?: string;
  type?: "text" | "email" | "password";
}

export const Input = React.forwardRef<HTMLInputElement, InputProps>(
  ({ label, error, type = "text", disabled, className = "", id, ...props }, ref) => {
    const defaultId = useId();
    const inputId = id || defaultId;
    const errorId = `${inputId}-error`;

    const containerClassNames = [
      styles.container,
      disabled && styles.disabled,
      error && styles.hasError,
      className
    ].filter(Boolean).join(" ");

    return (


        {label && (

            {label}

        )}

          [redacted]

        {error && (


              {error}


        )}


    );
```

```
  }
);

Input.displayName = "Input";
```

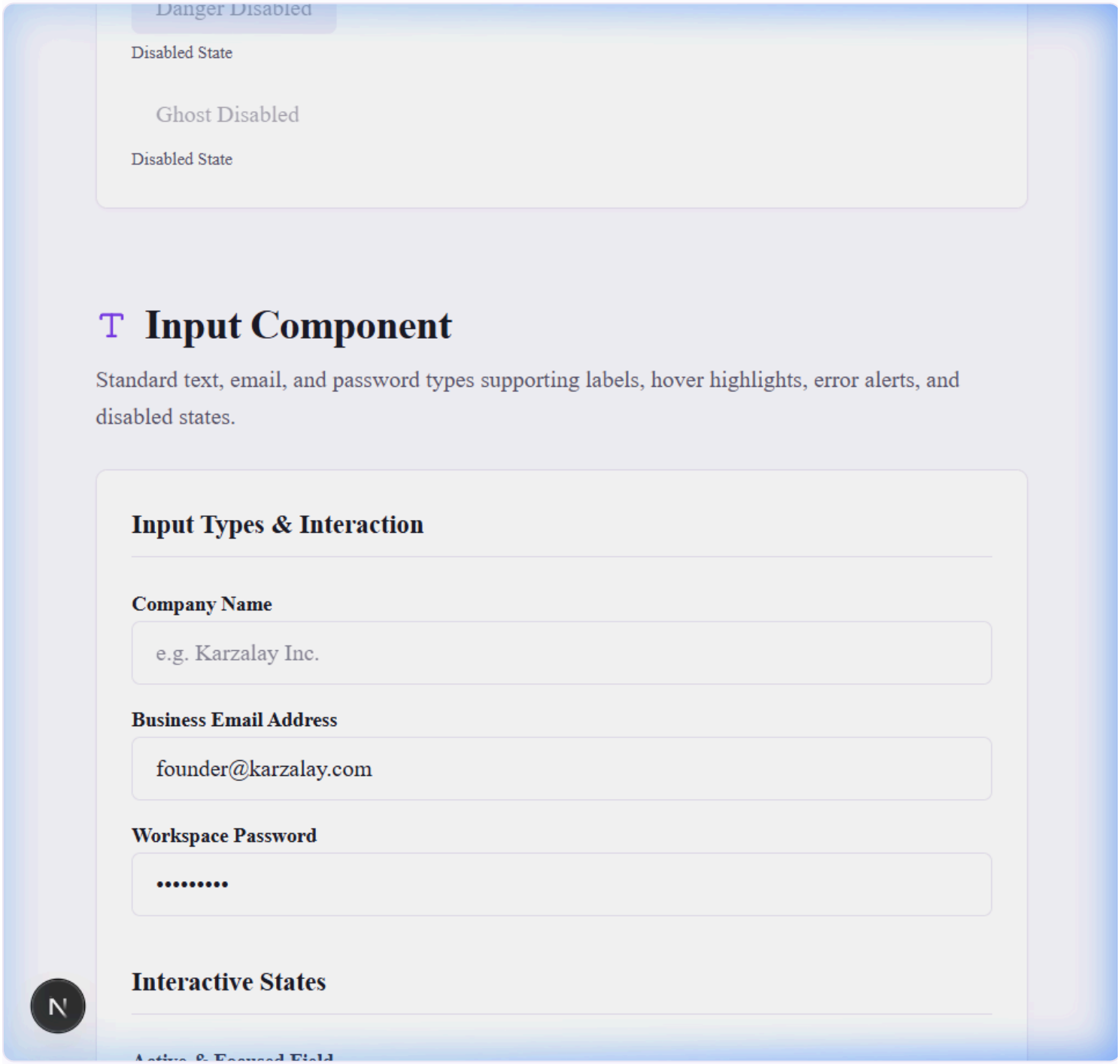

Figure 2.1: Input Types (Text, Email, Password) with label positions and placeholders

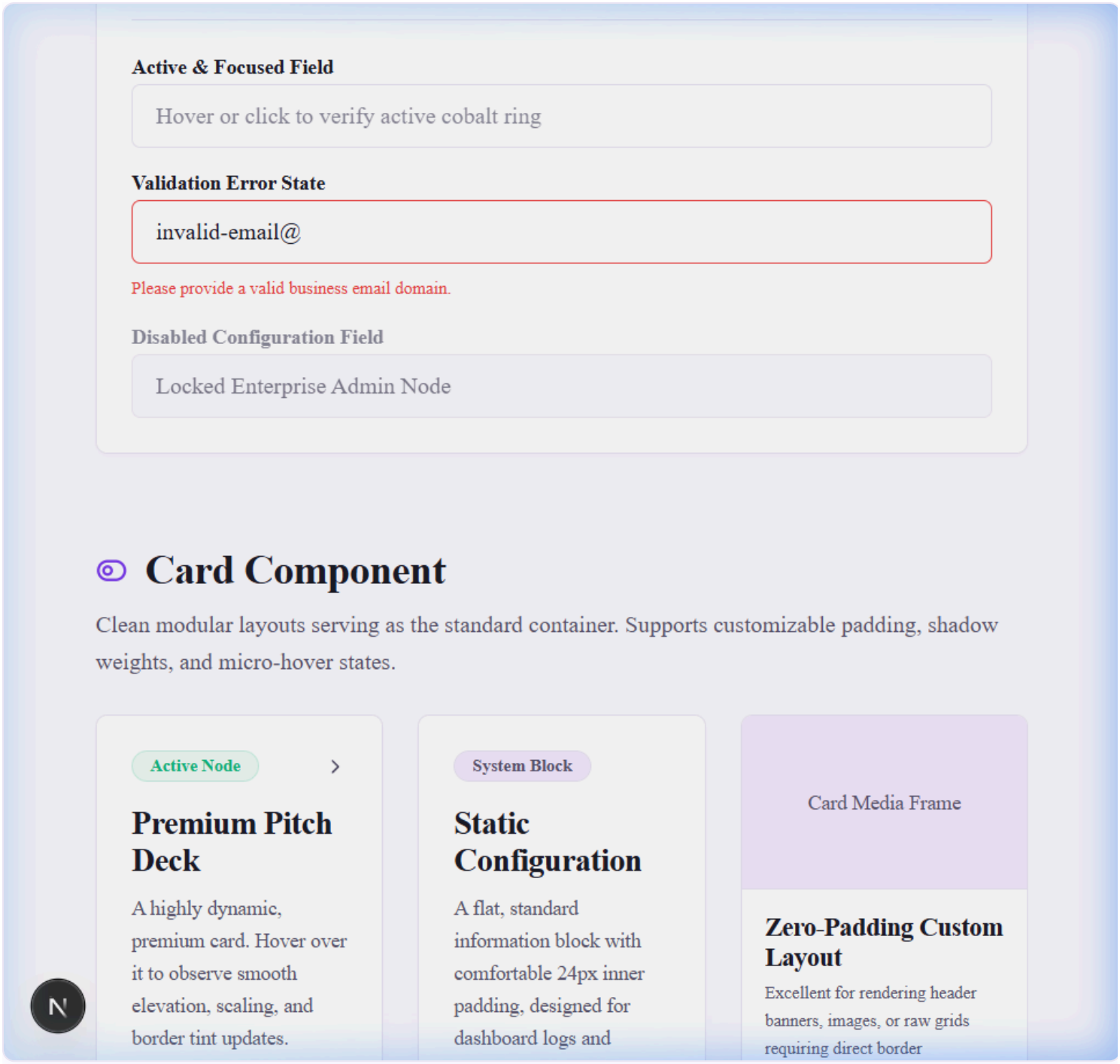

Figure 2.2: Active focus rings, validation error message indicators, and disabled overlay styles

📦 3. Card Component

Clean modular layout serving as the standard container for profile blocks, configuration nodes, and ticket streams throughout the application.

Props Accepted

Prop Name	Type	Default	Description
`padded`	`boolean`	`true`	When true, applies comfortable inner padding scales (`var(--spacing-24)`).
`shadow`	`boolean`	`true`	When true, applies design elevation shadows (`var(--shadow-card)`).
`hoverable`	`boolean`	`false`	Triggers micro-elevation transformations and shadow weights on hover.
`children`	`React.ReactNode`	—	Embedded visual layouts inside card structure.
`className`	`string`	`""`	Extra CSS classes to append.

Visual Variations

1. **Standard Flat Block:** Padded container with soft borders and light card shadows.
2. **Hoverable Dynamic Card:** Renders smooth scaling transitions, shifts upwards by `4px`, tints border to primary amethyst, and deepens drop-shadow weight.
3. **Zero Padding Layout:** Standard borders and layout bounds without paddings, designed for embedding full media banners or border-aligned lists.

React Implementation

```
import React from "react";
import styles from "./Card.module.css";

export interface CardProps extends React.HTMLAttributes<HTMLDivElement> {
  padded?: boolean;
  shadow?: boolean;
  hoverable?: boolean;
  children: React.ReactNode;
}

export const Card = React.forwardRef<HTMLDivElement, CardProps>(
  ({ padded = true, shadow = true, hoverable = false, className = "", children, ...props }, ref) => {
    const classNames = [
      styles.card,
      padded && styles.padded,
      shadow && styles.shadow,
      hoverable && styles.hoverable,
      className
    ].filter(Boolean).join(" ");

    return (


        {children}


    );
  }
);

Card.displayName = "Card";
```

🏷️ 4. Badge Component

Used for status indicators (Not Started, In Progress, Done, Blocked) and role labels. Accepts a color/variant prop. Pill-shaped with small text.

Props Accepted

Prop Name	Type	Default	Description
`variant`	`'neutral' | 'primary' | 'success' | 'warning' | 'danger' | 'info'`	`'neutral'`	Theme category selector mapping status/role context.
`children`	`React.ReactNode`	—	Text or icons embedded in pill container.
`className`	`string`	`""`	Extra CSS classes to append.

Variant Mapping Guidelines

- `neutral`: Standard info tags, inactive metadata, and "Not Started" states.
- `warning`: Represents amber statuses, alert triggers, or **In Progress** items.
- `success`: Represents green **Done** statuses, validated entries, or active nodes.
- `danger`: Represents red **Blocked** warnings, critical priorities, or errors.
- `primary`: Deep brand amethyst highlight tag mapping specialized roles (e.g. *Founder*).
- `info`: Soft slate secondary tagging for system credentials (e.g. *Super Admin*).

React Implementation

```
import React from "react";
import styles from "./Badge.module.css";

export interface BadgeProps extends React.HTMLAttributes<HTMLSpanElement> {
  variant?: "neutral" | "primary" | "success" | "warning" | "danger" | "info";
  children: React.ReactNode;
}

export const Badge = React.forwardRef<HTMLSpanElement, BadgeProps>(
  ({ variant = "neutral", className = "", children, ...props }, ref) => {
    const classNames = [
      styles.badge,
      styles[variant],
      className
    ].filter(Boolean).join(" ");

    return (

        {children}

    );
  }
);

Badge.displayName = "Badge";
```

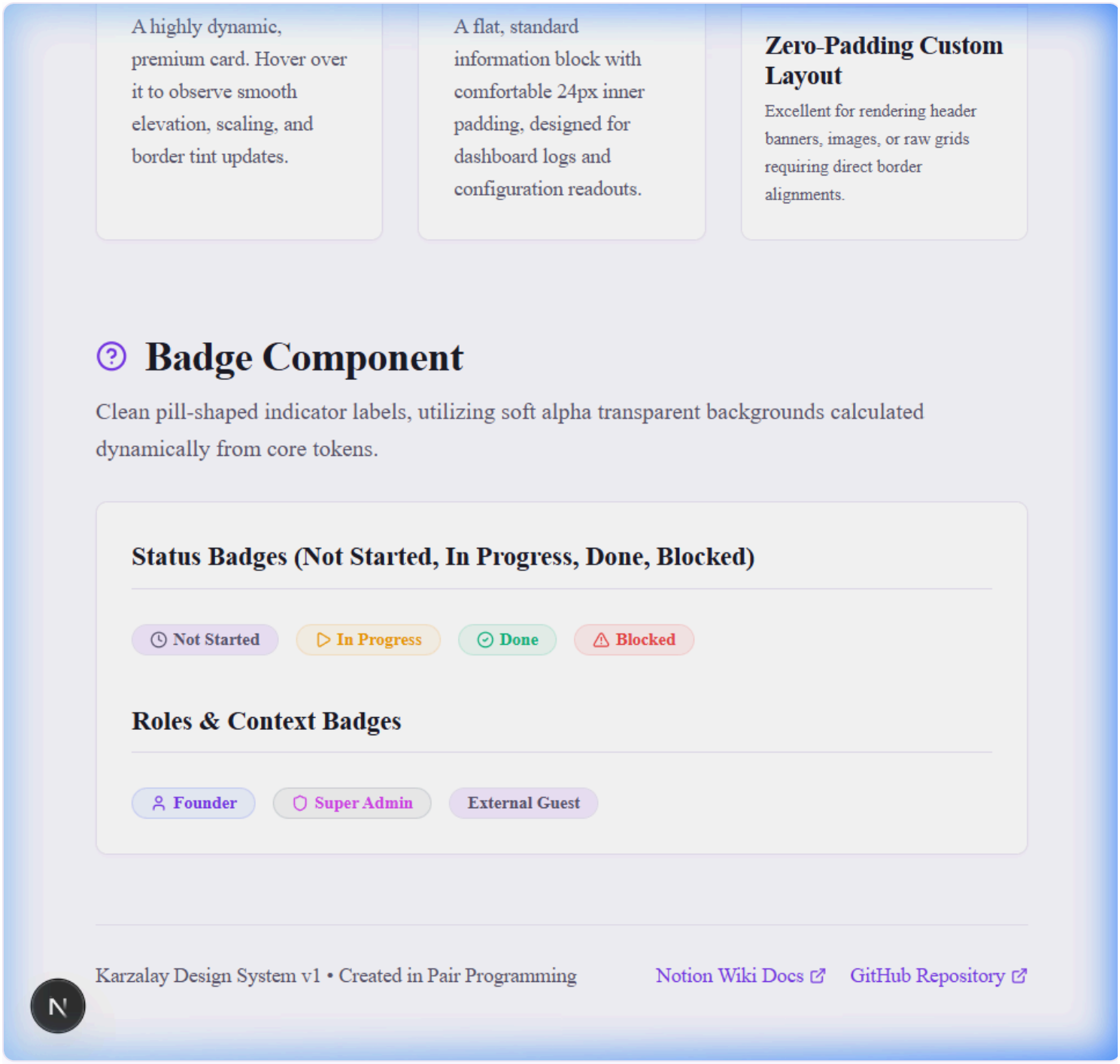

Figure 3.1: Card Variations (Hoverable, Static Flat, Zero Padding Banner) and Badge presets (Status, Roles)

🔬 5. Visual Verification & Staging Summary

All elements have been verified on the staging environment under a unified design profile. The staging verification route is live at `/test-components`.

- **Verification Checklist:**
  - [x] Zero hardcoded color palettes (all bound to CSS variable tokens).

- [x] Zero static font sizes (scales map properly from `tokens.css`).
- [x] Sizing adjustments preserve component boundaries.
- [x] Perfect accessibility compliance (labeled inputs, semantic outline rings, visible warning details, and disable layers).